

Tezcatl : principe, fonctions, vision

Tezcatl signifie « miroir » en nahuatl. Le nom vient de Tezcatlipoca, le miroir fumant. C'est l'ambition du projet en un mot. Tenir un miroir devant un système en production, et lui faire dire ce qui a changé.

Le problème

Une application en production produit deux flux d'information. Des logs, les messages texte qu'elle écrit en continu, et des métriques, des valeurs numériques mesurées régulièrement comme le temps de réponse ou la mémoire consommée. Les outils habituels du domaine stockent ces flux et permettent de les fouiller. Prometheus stocke les métriques, Loki stocke les logs, Grafana les affiche.

Aucun de ces outils ne répond à la question qu'on se pose vraiment à 14h07 quand le service se dégrade : qu'est-ce qui a changé, depuis quand, et quelles observations le prouvent. Cette synthèse, c'est un humain qui la fait, en ouvrant quatre onglets et en comparant des horodatages à la main.

Une chronologie d'incident typique ressemble à ça :

```
14:02  Déploiement d'une nouvelle version de checkout
14:04  Le temps de réponse monte
14:05  Un message d'erreur jamais vu apparaît : database connection timeout
14:06  Le pool de connexions à la base sature
```

Chaque ligne vit dans un outil différent. Le travail de l'ingénieur d'astreinte consiste à reconstituer ce tableau. Tezcatl est construit pour produire ce tableau tout seul.

Le principe

Le pari central est qu'un moteur déterministe et explicable suffit pour l'essentiel du travail. Pas de réseau de neurones, pas de modèle de langage dans la boucle de détection. Trois mécanismes simples, composés.

Apprendre la forme des logs. Un flux de logs brut contient en réalité peu de phrases distinctes. `user alice logged in` et `user bob logged in` sont le même événement, `user <*> logged in`, où `<*>` marque la partie variable. Ces squelettes de messages s'appellent des templates, et un algorithme publié en 2017, Drain, sait les découvrir au fil de l'eau, sans liste préétablie. Tezcatl en embarque une réimplémentation en Go, vérifiée en continu : l'intégration continue rejoue un même corpus de logs dans l'implémentation de référence et dans la nôtre, et exige un résultat identique ligne à ligne.

Apprendre le comportement normal. Une fois les logs réduits à des templates, on peut compter. À quelle fréquence chaque template apparaît, autour de quelle valeur chaque métrique oscille. Ces références tiennent compte de l'heure de la journée, pour que la sauvegarde nocturne et le pic de trafic de 9h ne déclenchent rien. Les détecteurs qui s'appuient dessus sont volontairement élémentaires. Un template jamais vu, un template devenu anormalement fréquent ou silencieux, une métrique qui s'écarte trop de sa moyenne ou franchit un seuil configuré. Chaque alerte embarque son propre calcul, valeur observée, référence, écart. Si un opérateur ne peut pas la vérifier à la main, elle ne vaut rien.

Corréler dans le temps. Les alertes d'un même service émises dans une fenêtre de 30 secondes fusionnent en un seul événement. L'événement contient les signaux regroupés, un score de confiance, les logs qui précèdent et suivent l'anomalie, et les déploiements récents. Un déploiement n'est jamais présenté comme une cause. C'est un fait daté, « 3 minutes avant », et l'interprétation reste humaine.

Le résultat est une ligne JSON, lisible par un humain comme par un programme :

```
{
  "kind": "anomaly.correlated",
  "service": "checkout",
  "severity": "critical",
  "confidence": 0.92,
  "summary": "new log template after learning period: database connection
timeout",
  "signals": ["log.new_template", "metric.threshold"],
  "related_changes": [{"change": {"type": "deployment", "version":
"checkout:v1.8.2"}, "offset_seconds": -180}],
  "context": {"before": ["..."], "after": ["..."]}
}
```

C'est la chronologie de tout à l'heure, assemblée par la machine.

Les fonctions, concrètement

Un binaire unique de 18 Mo, écrit en Go, sans rien à installer d'autre. Trois modes d'exécution.

Le mode standalone lit les logs sur son entrée standard et écrit les événements sur sa sortie. On branche l'outil comme un filtre shell :

```
journalctl -fu mon-app | tezcatl standalone logs --service mon-app
```

C'est le mode du diagnostic ponctuel. Le parseur accepte les logs tels qu'ils existent vraiment : JSON, sortie systemd, préfixes ajoutés par Docker ou Dokku, codes couleur inclus.

Le mode serveur centralise. Des clients légers, le même binaire, lui transmettent les logs de plusieurs machines par le réseau, en clair ou chiffré. Le serveur peut aussi interroger lui-même un Prometheus existant pour obtenir les métriques, sans agent à déployer.

Le mode replay rejoue un incident passé. Les fenêtres de corrélation avancent alors sur les horodatages des logs et non sur l'horloge du mur, donc rejouer six heures d'incident en dix secondes produit une chronologie exacte. C'est l'outil de validation du projet. Prendre trois incidents connus, les rejouer, vérifier que tezcatl rassemble ce qu'on avait cherché à la main.

Autour du moteur, le nécessaire pour vivre avec :

- Tout ce qui a été appris est sauvegardé sur disque et restauré au démarrage. Un redémarrage ne provoque ni réapprentissage ni vague de fausses alertes.

- Corriger l'outil est une commande. `tezcatl templates` liste ce qu'il a appris, `tezcatl mark --as ignore` fait taire un message bruyant, à chaud, et la correction survit aux redémarrages.
- Les événements partent sur la sortie standard, dans PostgreSQL, ou vers un webhook pour notifier. Une destination en panne ne bloque jamais l'analyse, elle perd des événements et le compte.
- Le moteur traite environ 120 000 lignes par seconde sur un cœur, détection comprise. Les déploiements visés en produisent quelques centaines. La marge est confortable.

Ce que tezcatl n'est pas

Pas un remplaçant de Prometheus ou de Loki. Tezcatl ne stocke ni métriques ni logs bruts au-delà de ses fenêtres de contexte. Il se place au-dessus des données existantes et n'en garde que le sens.

Pas un produit d'AI Ops au sens marketing. Aucune promesse d'analyse de cause racine automatique. Le corrélateur dit « ces faits sont proches dans le temps », jamais « ceci a causé cela ».

Pas un agent IA. C'est le choix structurant du projet, et il est contre-intuitif en 2026. Un modèle de langage qui lirait les flux bruts coûterait cher, répondrait lentement et inventerait des causes avec aplomb. Le plan est inverse. Le moteur déterministe compresse des millions de lignes en quelques événements denses et vérifiables, et c'est ce petit résumé, éventuellement, qu'un modèle recevra un jour pour rédiger une explication. La détection reste du côté vérifiable de la frontière.

La vision

Le projet avance en s'utilisant lui-même. D'abord des applications personnelles réelles, hébergées sur Dokku et systemd, où le mode léger rend déjà service. Ensuite une application Kubernetes qui souffre d'un vrai problème de diagnostic. Le critère de réussite est resté le même depuis le premier jour. Est-ce que tezcatl rassemble automatiquement ce qu'un ingénieur cherche aujourd'hui à la main dans plusieurs outils. Si oui, le projet a une valeur pratique démontrable, même sans innovation algorithmique. Je préfère cette barre-là à un benchmark académique.

La suite est ordonnée par ce critère. Relier entre eux les services touchés par un même incident, là où l'analyse est aujourd'hui service par service. Structurer l'événement pour distinguer le déclencheur, les preuves et les hypothèses. Capturer automatiquement les déploiements Kubernetes et Dokku. Et en dernier, l'export vers un modèle de langage, quand les événements auront prouvé leur qualité sur des incidents réels.

Le miroir d'abord. L'interprète ensuite.